Facultad de Ingeniería

Carrera Profesional de Ingeniería de Software

Inversión de prioridades en el RTOS de Mars Pathfinder

Por qué falló, cómo falló y cómo se corrigió

Trabajo Final de Sistemas Operativos

Asignatura: Sistemas Operativos

Docente: Mg. Maribel Molina Barriga

Integrantes:
- Diego Eduardo Zapana Choquehuayta
- Heydy Anyely Castillo Arocutipa
- Angelica Giovanna Huaman Cuba
- Grecia Anhi Mamani Larico

Arequipa, Perú

2 de noviembre de 2026

Inversión de prioridades en el RTOS de Mars Pathfinder: por qué falló, cómo falló y cómo se corrigió

***Resumen*—Mars Pathfinder aterrizó en Marte el 4 de julio de 1997 y desplegó el rover Sojourner. Días después, el ordenador de vuelo del módulo de aterrizaje comenzó a reiniciarse. Este artículo reconstruye el caso con fuentes primarias y un modelo determinista en C11 y ensamblador x86-64. El análisis muestra que el fallo no fue un error del planificador ni una pérdida de datos, sino una inversión de prioridades no acotada en el mutex interno de `select()`, configurado sin herencia de prioridad y agravado por una carga de tareas medias superior a la prevista. El watchdog de `bc_sched` convirtió el incumplimiento del ciclo de 1553, de 125 ms, en un reinicio. El modelo reproduce el fallo: con la herencia desactivada se producen tres reinicios y un bloqueo máximo de 125 ms; al activarla, los reinicios desaparecen y el bloqueo baja a 5 ms; un control sin mutex compartido confirma la causa. La conclusión es que la corrección temporal es una propiedad de configuración y de observabilidad, no solo del algoritmo de planificación.**

***Palabras clave*—Sistemas operativos de tiempo real, inversión de prioridades, herencia de prioridad, Mars Pathfinder, VxWorks, watchdog.**

I. Introducción

El 4 de julio de 1997, la misión Mars Pathfinder aterrizó en Ares Vallis y desplegó el rover Sojourner, el primer robot autónomo en recorrer la superficie de Marte [1]. La misión se presentó como un éxito tecnológico: el aterrizaje con airbags, el despliegue del rover y la transmisión de imágenes y datos científicos funcionaron según lo previsto. Sin embargo, pocos días después del aterrizaje, el ordenador de vuelo del módulo de aterrizaje comenzó a reiniciarse de forma intermitente [2]. Cada reinicio interrumpía las actividades comandadas desde Tierra y retrasaba la recolección de datos.

La prensa describió el problema como un "fallo de software" o como un ordenador "que intentaba hacer demasiadas cosas a la vez". La explicación técnica es más precisa y más interesante: el sistema no fallaba al azar. Una tarea de baja prioridad, `ASI/MET`, tomaba un mutex interno del mecanismo `select()` y era expropiada antes de liberarlo; una tarea de alta prioridad, `bc_dist`, quedaba bloqueada esperando ese mutex; las tareas de prioridad media se ejecutaban mientras tanto; y el watchdog de `bc_sched`, al comprobar que `bc_dist` no había terminado dentro del ciclo de 1553, ordenaba un reinicio [2].

El caso se cita con frecuencia en libros y cursos, pero casi siempre en un párrafo: "Pathfinder sufrió una inversión de prioridades y se arregló activando la herencia de prioridad". Esa síntesis es correcta, pero deja fuera tres preguntas que este artículo considera inseparables: *por qué* falló, *cómo* falló y *cómo* se corrigió. Responderlas exige reconstruir la cadena causal completa, distinguir la causa inmediata de las condiciones que la hicieron posible y comprobar el contrafactual: si la herencia de prioridad se activa, ¿desaparece el fallo?

La reconstrucción no está exenta de dificultades. La cuenta más citada es un correo que Reeves escribió para corregir las versiones periodísticas del incidente, no un artículo arbitrado; su valor es el de un testimonio directo del responsable del software de vuelo [2]. La versión de 1999 añade el diagnóstico y la corrección, pero tampoco es una auditoría independiente [3]. Por eso el artículo contrasta ambas cuentas con los datos de misión de NASA y con una nota de prensa contemporánea, y separa lo documentado de lo inferido [1], [4]. Esa separación es importante: el caso se presta a simplificaciones que atribuyen el fallo al rover, a una pérdida de datos o a un error del planificador, y ninguna de las tres es correcta.

La relevancia del caso no es histórica. En una encuesta a 120 profesionales de sistemas de tiempo real, el 78,4 % declaró usar un RTOS o microkernel, el 55,9 % usaba Linux y el 62,7 % combinaba varios sistemas operativos; la planificación por prioridad fija era la política más usada (56,25 %), mientras que EDF solo aparecía en el 16,67 % [5]. Además, en un cuestionario sobre esos mismos resultados, los investigadores académicos acertaron el 27 %, por debajo del azar (33,3 %), y los profesionales de la industria el 35,1 % [5]. La brecha entre lo que se investiga y lo que se practica sigue abierta, y la clase de fallo que afectó a Pathfinder —bloqueo no acotado por un recurso compartido— no ha desaparecido.

Este trabajo se organiza alrededor de tres preguntas de investigación:

- **RQ1.** ¿Cuál fue la causa raíz del fallo y qué condiciones del sistema la hicieron posible?
- **RQ2.** ¿Cómo se manifestó en la temporización del ciclo de 1553 y qué papel jugó el watchdog?
- **RQ3.** ¿Qué corrección se aplicó, por qué ataca la causa y qué evidencia demuestra que funcionó?

Las contribuciones son tres. **C1:** una reconstrucción causal del caso con una cadena de evidencia que separa fuentes primarias de secundarias. **C2:** un modelo determinista, reproducible, que reproduce el fallo y su contrafactual. **C3:** un conjunto de lecciones de diseño y verificación para RTOS actuales. El resto del artículo se organiza así: la Sección II presenta el marco teórico y el estado del arte; la Sección III describe el método y el modelo; la Sección IV desarrolla el análisis y la discusión; la Sección V concluye.

II. Marco teórico y estado del arte

II-A. Tiempo real y planificación de prioridad fija

Un sistema operativo de tiempo real no se define por su velocidad, sino por su capacidad de garantizar que una tarea termine antes de su plazo. Una tarea periódica se caracteriza por su período T_i, su peor tiempo de ejecución C_i y su plazo relativo D_i; el conjunto es planificable si existe un orden de ejecución que cumple todos los plazos en el peor caso. En la planificación por prioridad fija, cada tarea recibe una prioridad estática y el procesador ejecuta siempre la tarea lista de mayor prioridad. Rate Monotonic (RM) asigna prioridades inversamente proporcionales al período y es óptimo entre los algoritmos de prioridad fija para tareas periódicas con plazos iguales a los períodos [6]. Earliest Deadline First (EDF) es dinámico y óptimo para uniprocesador con utilización menor o igual a uno, pero exige conocer los plazos en tiempo de ejecución.

Los sistemas de tiempo real combinan dos enfoques. En el enfoque *event-triggered*, la decisión de planificación se toma en ejecución y el análisis se basa en cotas de tiempo de respuesta; en el enfoque *time-triggered*, el horario se sintetiza en tiempo de diseño y el sistema lo sigue por construcción [7]. La planificación time-triggered es muy predecible y facilita la certificación, pero la síntesis del horario es un problema NP-hard en su forma general [7]. Pathfinder usaba un esquema híbrido: un ciclo periódico de 8 Hz para el bus 1553 y planificación por prioridad fija para las tareas del software de vuelo [2].

La planificación de prioridad fija garantiza plazos solo si el tiempo de bloqueo está acotado. En un sistema con recursos compartidos, una tarea de alta prioridad puede esperar a que una tarea de baja prioridad libere un mutex; si nada limita esa espera, el análisis de planificabilidad deja de ser válido. La contención no es un detalle de implementación: es una fuente de interferencia que debe modelarse. Guasque et al. muestran que las cotas de planificabilidad para prioridades dinámicas en multicore cambian de forma significativa cuando se incorpora la contención, y que la elección del algoritmo de asignación de tareas modifica la fracción de sistemas que resulta planificable [8].

Para tareas periódicas con plazos iguales a los períodos, RM ofrece una garantía sencilla: el conjunto es planificable si la utilización $U = \sum_i C_i/T_i$ no supera la cota $n(2^{1/n} - 1)$, que tiende a $\ln 2$ cuando n crece [6]. La cota es suficiente, no necesaria, y por eso los análisis más ajustados —como los de Guasque et al. para prioridades dinámicas— son valiosos: permiten admitir sistemas que la cota clásica rechaza. Pathfinder no era un caso de RM puro, pero la lógica es la misma: la garantía depende de que el bloqueo y la interferencia estén acotados.

II-B. Recursos compartidos, bloqueo e inversión de prioridades

Un mutex es un mecanismo de exclusión mutua: una tarea lo adquiere antes de entrar en una sección crítica y lo libera al salir. Si una tarea de alta prioridad intenta adquirir un mutex que ya posee una tarea de baja prioridad, la primera queda bloqueada. El problema aparece cuando, entre ambas, se ejecutan tareas de prioridad intermedia: la tarea baja no puede avanzar para liberar el mutex, la tarea alta sigue bloqueada y el tiempo de bloqueo deja de estar acotado por la duración de la sección crítica. Ese fenómeno se denomina *inversión de prioridades* [9].

La herencia de prioridad (PIP) corrige el problema: cuando una tarea de alta prioridad se bloquea en un mutex, la tarea que lo posee hereda temporalmente la prioridad de la tarea bloqueada. Así, las tareas intermedias no pueden expropiarla y el bloqueo queda acotado por la sección crítica. El protocolo de techo de prioridad (PCP) ofrece una garantía similar mediante un techo estático por recurso [9]. La diferencia entre "bloqueo acotado" y "bloqueo no acotado" es la diferencia entre un sistema planificable y un sistema que falla solo bajo ciertas cargas.

En los RTOS comerciales, la herencia de prioridad suele ser una opción de configuración del mutex, no un comportamiento automático. Esa decisión de diseño es relevante: un mutex creado sin herencia puede pasar inadvertido durante años si la carga no alcanza el escenario crítico. Dong et al. muestran que el problema reaparece en fronteras entre un RTOS y un GPOS: su protocolo RTRG-RPC usa un *priority-swapping* inspirado en herencia de prioridad para evitar que la tarea contenedora del GPOS bloquee a la tarea de tiempo real [10]. La inversión de prioridades no es un accidente de un sistema antiguo; es una propiedad estructural de los sistemas con recursos compartidos y prioridades.

La inversión requiere tres condiciones: una tarea de alta prioridad, una de baja que posee un recurso y una o más tareas intermedias que puedan ejecutarse mientras la alta espera. Si falta cualquiera de las tres, el bloqueo se acota por la sección crítica o por la ausencia de interferencia. Esa es la razón por la que el fallo de Pathfinder dependía de la carga: sin tareas medias suficientes, la tarea baja avanzaba y liberaba el mutex antes de que el plazo venciera. La herencia de prioridad elimina la tercera condición al impedir que las intermedias expropien a la poseedora del recurso.

II-C. Estado del arte 2020–2025

La literatura reciente se puede agrupar en tres bloques. El primero es el análisis de planificabilidad y la gestión de recursos. Minaeva y Hanzálek sistematizan la planificación periódica time-triggered y muestran que la representación del problema determina su complejidad [7]. Guasque et al. proponen dos cotas basadas en la *demand bound function* para prioridades dinámicas con contención; la cota menos pesimista se acerca más a la utilización real, y el asignador que considera la interferencia (Wmin) es el más preciso [8]. He et al. abordan el desperdicio de la asignación rectangular de *federated scheduling* con una asignación "tipo escalera" que recupera hasta un 48,3 % más de recursos y un 37,8 % en promedio, sin romper las garantías duras [11]. Salman et al. estudian tareas elásticas bajo reservas de paralelismo mínimo en multiprocesador y concluyen que su enfoque es competitivo en promedio, aunque ningún método domina en el peor caso [12].

El segundo bloque son los mecanismos del sistema operativo. ReTiF ofrece una API declarativa para que aplicaciones sin privilegios accedan al scheduling de tiempo real de un POSIX, con plugins para EDF, RM y round-robin, y un modelo de control de acceso que media las peticiones [13]. Su evaluación muestra que la mediación añade una sobrecarga de unos 12 μs en Intel y 50 μs en ARM, aceptable fuera de los bucles críticos, y que el framework no implementa análisis de bloqueo [13]. Dong et al. construyen el ya mencionado RTRG-RPC sobre TrustZone y descomponen su tiempo de respuesta en cinco términos, lo que permite razonar sobre la latencia de la frontera entre dominios [10]. Manjunath y Baunach reutilizan resultados intermedios de WCET/OTAWA para verificar código RTOS de bajo nivel; detectan dependencias de sincronización fallidas y carreras de memoria en el cambio de contexto de un AUTOSAR y en el arranque de FreeRTOS sobre AURIX TriCore [14].

El tercer bloque es la plataforma, la observabilidad y la práctica. Farina et al. usan Intel RDT para detectar y regular la interferencia de memoria en sistemas de nube de tiempo real: la ocupación de la Read Pending Queue permite estimar cuántos núcleos acceden a memoria y la regulación MBA limita el ancho de banda [15]. Timerlat, integrado en el kernel Linux desde la versión 5.17, mide por separado la latencia de IRQ, de hilo y de usuario, y añade un autoanálisis que señala las causas de un pico [16]. En su evaluación, un sistema con PREEMPT_RT y aislamiento de núcleos reduce la latencia máxima de decenas de milisegundos a 58 μs, y a 10 μs en el mejor caso [16]. Akesson et al. cierran el bloque con la encuesta industrial ya citada: la práctica combina varios SO, prioridad fija y planificación cíclica, y la predictibilidad temporal no es el único criterio de diseño [5].

Dos detalles de ese bloque merecen énfasis. Timerlat no se limita a medir: descompone la latencia en IRQ, hilo y usuario, y su autoanálisis identifica la causa probable de un pico —una interrupción, una sección crítica, una migración— sin exigir que el usuario sea experto en trazas [16]. Farina et al. muestran que la interferencia de memoria se puede detectar con un contador de ocupación de la cola de lecturas y regular con MBA, pero que la relación entre el retardo configurado y el ancho de banda efectivo no es lineal: solo ciertos valores producen una limitación significativa [15]. Ambos trabajos comparten una idea: lo que no se mide no se puede garantizar.

II-D. Síntesis

El corpus muestra que la investigación reciente se concentra en tres frentes: cotas más ajustadas, mecanismos de acceso y aislamiento, y herramientas de observabilidad y verificación. Pathfinder es anterior a todos ellos, pero pertenece a la misma clase de problema: un bloqueo no acotado por un recurso compartido que se manifiesta solo bajo una carga concreta. La diferencia es que el caso permite reconstruir la cadena completa —causa, manifestación, corrección— con fuentes primarias y un contrafactual ejecutable. Esa es la contribución que este artículo busca aportar.

Tabla I
CADENA DE EVIDENCIA DEL CASO.

Afirmación	Fuente	Tipo
Aterrizaje y despliegue del rover	NASA [1]	Primaria
Tareas, prioridades y ciclo de 8 Hz	Reeves 1997 [2]	Primaria
Secuencia del bloqueo y reset	Reeves 1997 [2]	Primaria
Diagnóstico en menos de 18 h	Reeves 1999 [3]	Primaria
Corrección con herencia de prioridad	Reeves 1999 [3]	Primaria
Fecha del parche (19/7/1997)	AP 1997 [4]	Secundaria

III. MÉTODO

III-A. Diseño del estudio

Se adoptó un estudio de caso instrumental con una componente experimental. El caso es el fallo del ordenador de vuelo del lander Mars Pathfinder; la unidad de análisis es el conjunto de tareas del software de vuelo durante el ciclo de bus de 125 ms. El estudio combina tres fuentes: documentos primarios del caso, literatura técnica sobre tiempo real y un modelo determinista del mecanismo. El modelo no es una emulación de VxWorks: es una reconstrucción propia, escrita en C11 y ensamblador x86-64, que reproduce las relaciones causales relevantes con tiempos simulados.

III-B. Fuentes y cadena de evidencia

La fuente principal es la cuenta de Glenn E. Reeves, responsable del software de vuelo de JPL, en su correo del 15 de diciembre de 1997 [2] y en su artículo de *Dr. Dobb's Journal* de noviembre de 1999 [3]. El correo describe la arquitectura, las prioridades, la secuencia del bloqueo y el reset; el artículo amplía el diagnóstico, la corrección y las dudas sobre el impacto del cambio. Los datos de misión se tomaron de la página oficial de NASA [1]. La fecha del parche se corroboró con una nota de prensa contemporánea [4]. La Tabla I resume la cadena.

Se distingue entre hechos documentados y inferencias. Por ejemplo, que el mutex de `select()` se creara sin herencia de prioridad es un hecho documentado; que la carga de tareas medias fuera mayor de lo previsto es una condición descrita por Reeves y consistente con el contexto de la misión; que el fallo se manifestara solo en ciertos soles es una inferencia a partir de la intermitencia observada. El artículo marca esas diferencias en el texto.

III-C. Modelo determinista

El modelo se implementó en `entornoRTOS/` con C11 y ensamblador x86-64. El cambio de contexto se realiza en `src/switch.S`; no se usa `ucontext` ni ninguna biblioteca externa. El tiempo es simulado y determinista: no hay reloj de pared, aleatoriedad ni dependencia de la carga del host. La Tabla II resume las tareas y prioridades.

El mutex se crea con o sin la opción de herencia de prioridad. Cuando está activa, la tarea que posee el mutex hereda la

Tabla II
TAREAS DEL MODELO Y PRIORIDADES.

Tarea	Prior.	Activación	Recurso
`bc_sched`	1	cada 125 ms	bus 1553
`bc_dist`	3	tras el bus	`select_mutex`
`med_1..3`	4–6	periódica	CPU
`ASI/MET`	7	periódica	`select_mutex`
`tExec`	idle	—	CPU ociosa

prioridad de la tarea bloqueada y la recupera al liberarlo. El modelo incluye `pipe()`, un `fd_set` y un `select()` con su mutex interno, para reproducir la interacción exacta que describió Reeves. El bus 1553 se modela como un evento periódico de 125 ms; `bc_sched` comprueba al inicio de cada ciclo si `bc_dist` terminó y, si no, registra un fallo de plazo y ordena un reset.

El scheduler es expropiativo y de prioridad fija. Cada cambio de contexto se realiza en `switch.S`, que guarda y restaura el estado de los registros según la convención System V; no se usa `ucontext` ni una biblioteca de hilos. El tiempo avanza por eventos: el planificador salta al siguiente instante relevante —una activación, un vencimiento de temporizador, una liberación de mutex— y no ejecuta un bucle de *ticks*. Esa decisión mantiene el modelo determinista y permite simular dos segundos de vuelo en una fracción de segundo. La traza registra cada evento con su instante, la tarea, la prioridad efectiva y el recurso; el visualizador la usa para reconstruir la línea de tiempo.

III-D. Escenarios y métricas

Se definieron cuatro escenarios. S0 representa la carga baja de las pruebas previas al lanzamiento, con la herencia desactivada. S1 representa la carga alta posterior al aterrizaje, con la herencia desactivada: es el escenario del fallo. S2 repite S1 con la herencia activada: es el contrafactual de la corrección. S3 elimina el recurso compartido —`ASI/MET` y `bc_dist` usan mutex separados— y funciona como control para aislar la causa.

Las métricas son: número de reinicios, tiempo de bloqueo máximo de `bc_dist`, número de fallos de plazo y utilización de CPU de las tareas medias. Todas se obtienen de la traza de eventos y se agregan en la Tabla IV. La reproducibilidad se verificó con `make test`, que comprueba las firmas de los escenarios, el bus y el cambio de contexto, y con `make run-pip-off` y `make run-pip-on`, que imprimen `RESET` y `OK` respectivamente.

III-E. Validación y límites

La validación tiene tres niveles. El primero es estructural: cada tarea, prioridad y período se justifica con Reeves o se declara como supuesto. El segundo es conductual: el modelo debe cumplir las cuatro firmas del caso (S0 sin reinicios, S1 con reinicios, S2 sin reinicios, S3 sin reinicios). El tercero es de reproducibilidad: el modelo es determinista y produce las mismas trazas en cada ejecución.

Las limitaciones son explícitas. El modelo no es VxWorks, no usa su código fuente ni su binario, no emula caché ni pipeline y no reproduce la telemetría de vuelo. Los tiempos son sintéticos y están calibrados para reproducir las relaciones causales, no para coincidir al microsegundo con el vuelo real. Por tanto, los resultados deben leerse como una confirmación del mecanismo, no como una medición del Pathfinder histórico.

IV. DESARROLLO Y DISCUSIÓN

IV-A. Por qué falló

La causa inmediata fue una inversión de prioridades no acotada. El mutex que `select()` crea para proteger la lista de descriptores se inicializó sin la opción de herencia de prioridad. VxWorks permite esa configuración y la usa por defecto por razones de rendimiento [2], [3]. Cuando `ASI/MET`, la tarea de menor prioridad, tomó el mutex y fue expropiada antes de liberarlo, cualquier tarea que intentara adquirirlo quedó bloqueada. `bc_dist`, de prioridad 3, fue esa tarea.

La condición habilitante fue la carga de las tareas medias. El fallo no se manifiesta con cualquier carga: necesita que `ASI/MET` esté dentro de la sección crítica, que `bc_dist` llegue al mutex y que las tareas de prioridad intermedia tengan suficiente trabajo para impedir que `ASI/MET` avance. Reeves describe que la antena funcionó mejor de lo previsto, que había más datos que transmitir y que las tareas de comunicaciones ocupaban más CPU de la estimada [2]. Sin esa carga, la inversión existe pero el plazo se cumple; con ella, el bloqueo supera los 125 ms.

La condición latente fue el proceso de prueba. Las pruebas previas al lanzamiento se hicieron con el “mejor caso” de tasas de datos y actividades científicas; el fallo se había observado una vez, pero no se pudo reproducir y se priorizó el software de aterrizaje [2]. No hubo una decisión consciente de desactivar la herencia: simplemente se pasó por alto el comportamiento interno del mecanismo `select()`/`pipe()`. La lección de Reeves es explícita: al usar software comercial, hay que conocer cómo funciona [2].

Desde la teoría, el resultado es el esperado. Sin herencia, el tiempo de bloqueo de una tarea alta por una baja no está acotado por la sección crítica: puede incluir todas las expropiaciones de las tareas intermedias. Con herencia, la tarea baja hereda la prioridad de la alta y las intermedias no pueden expropiarla; el bloqueo queda acotado por la duración de la sección crítica [9]. El fallo de Pathfinder no fue una violación de la teoría: fue una configuración que dejó el sistema fuera de las hipótesis de la teoría.

La causa se puede resumir en una cadena de tres niveles. El nivel inmediato es la opción del mutex; el nivel habilitante es la carga de tareas medias; el nivel latente es el proceso de prueba y la confianza en el valor por defecto. Los tres deben estar presentes para que el fallo se manifieste. Esa estructura explica por qué el caso es tan instructivo: no hay un único culpable que se pueda corregir con una línea, sino una combinación de decisiones que, por separado, parecen razonables.

IV-B. Cómo falló

La secuencia se puede reconstruir en siete pasos, que coinciden con la traza del modelo S1. La Tabla III los resume.

Tabla III
SECUENCIA DEL FALLO EN S1 (TIEMPOS DE LA TRAZA).

t (ms)	Evento	Estado
499,0	ASI/MET adquiere select_mutex	P7
500,0	bc_sched expropia a ASI/MET	P1
500,1	bc_dist intenta el mutex y se bloquea	P3
507,1	med_3 ejecuta una ráfaga de CPU	P6
625,0	bc_sched detecta que bc_dist no terminó	P1
625,0	bc_sched ordena el reset	—
733,2	ASI/MET libera el mutex (tras el reset)	P7

El ciclo normal del bus tiene cinco instantes: el bus arranca en el límite de 8 Hz, el tráfico de 1553 termina y despierta a `bc_dist`, `bc_dist` distribuye los datos, `bc_sched` se activa para preparar el ciclo siguiente y comprueba que `bc_dist` haya terminado. El fallo ocurre cuando esa comprobación falla. El reset no es un accidente: es la reacción prevista del sistema ante un incumplimiento de plazo. Reinicializa hardware y software, termina las actividades comandadas desde Tierra y retrasa el resto de la jornada; los datos ya recogidos en RAM se conservan si no se pierde la alimentación [2].

El impacto operativo es importante para interpretar el caso. No hubo una pérdida catastrófica de datos ni una destrucción del vehículo: hubo una pérdida de tiempo de misión. Cada reset interrumpía las actividades del día y obligaba a repetirlas. El sistema se recuperaba, pero no corregía la causa; el fallo era permanente y se repetía cada vez que se daba la combinación de carga. Esa diferencia entre *recuperación* y *corrección* es una de las lecciones centrales del caso.

Por eso el fenómeno parecía aleatorio. El reinicio no dependía de una tarea concreta ni de un instante fijo, sino de la coincidencia de tres eventos: que `ASI/MET` estuviera dentro de la sección crítica, que `bc_dist` llegara al mutex antes de que la primera lo liberara y que las tareas medias tuvieran suficiente trabajo para llenar la ventana. Cuando la coincidencia no se daba, el sistema funcionaba con normalidad. Esa intermitencia es la razón por la que las pruebas de “mejor caso” no lo detectaron.

El modelo S1 reproduce la secuencia: `ASI/MET` toma el mutex a los 499 ms, `bc_dist` se bloquea a los 500,1 ms, `med_3` ejecuta a los 507,1 ms y `bc_sched` detecta el fallo a los 625 ms. El bloqueo máximo es de 125 ms y se producen tres reinicios en los dos segundos simulados. La Fig. 1 muestra la diferencia entre S1 y S2.

IV-C. Cómo se solucionó

El diagnóstico fue posible gracias a una facilidad de traza que el equipo había dejado a bordo por diseño. La herramienta instrumentaba los servicios `pipe`, `msgQ`, el manejo de interrupciones, `select` y la tarea `tExec`; escribía en buffers circulares y volcaba la información cuando se detectaba el error [2], [3]. Con esa traza, el equipo reprodujo en la réplica de laboratorio el mismo conjunto de actividades y, en menos de 18 horas, aisló la interacción `pipe()`/`select()`, la diagnosticó como inversión de prioridades e identificó la corrección [3].

La corrección consistió en habilitar la herencia de prioridad en la creación del mutex de `select()`. La opción se controlaba mediante una variable global que afectaba a todos los semáforos creados por `selectLib()`; no había una forma sencilla de cambiarla solo para el mutex del problema [3]. El equipo evaluó cuatro riesgos: el efecto global del cambio, el posible impacto en el resto del sistema, la degradación de rendimiento y un posible cambio de comportamiento del propio `select()`. Wind River analizó los dos últimos y concluyó que el impacto sería mínimo y que el comportamiento no cambiaría mientras hubiera una sola tarea esperando por descriptor [3]. El cambio se probó, se aplicó y el problema se resolvió.

Las cuatro preocupaciones ilustran por qué la corrección no fue trivial. La opción se aplicaba a todos los semáforos creados por `selectLib()`, no solo al del `pipe()` problemático; el equipo no sabía si el cambio alteraría el comportamiento de otros dispositivos; la opción se había omitido por rendimiento, y no estaba claro cuánto se degradaría; y podía haber una interacción interna del propio `select()` con la herencia. Wind River respondió a las dos últimas y el equipo verificó las dos primeras con análisis y pruebas. La lección es que una corrección de una línea puede tener un alcance global y exige tanta validación como el fallo que corrige.

El modelo confirma el mecanismo. En S2, `ASI/MET` adquiere el mutex a los 499 ms, `bc_dist` se bloquea a los 500,1 ms, `ASI/MET` hereda la prioridad 3 y libera el mutex a los 502,1 ms; `bc_dist` termina a los 505,1 ms. El bloqueo máximo baja de 125 ms a 5 ms y no hay reinicios. En S3, donde `ASI/MET` y `bc_dist` usan mutex separados, el bloqueo máximo es de 3 ms y tampoco hay reinicios. La comparación aísla la causa: no es la carga por sí sola, sino el recurso compartido sin herencia.

IV-D. Validación y sensibilidad

La Tabla IV resume los cuatro escenarios. Los resultados son deterministas y se reproducen con `make all-scenarios`. La diferencia entre S1 y S2 es el contrafactual de la corrección; la diferencia entre S1 y S3 es el control que confirma que el recurso compartido es necesario para el fallo.

La validación estructural se apoya en la Tabla I: las tareas, prioridades, período y secuencia provienen de Reeves. La validación conductual se cumple: S0 no falla, S1 falla, S2 no falla y S3 no falla. La reproducibilidad se verificó con `make test`, que incluye pruebas de firmas, del bus y del cambio de contexto, y con la comparación de trazas entre ejecuciones.

El alcance de la confirmación es limitado. El modelo no demuestra que VxWorks se comportara exactamente así en Marte; demuestra que el mecanismo descrito por Reeves produce el fallo cuando se dan las condiciones y que la herencia de prioridad lo elimina. Es una confirmación causal, no una medición histórica. Tampoco se exploró un barrido completo de carga ni una sensibilidad sistemática a la duración de la sección crítica; esos experimentos quedan como trabajo futuro.

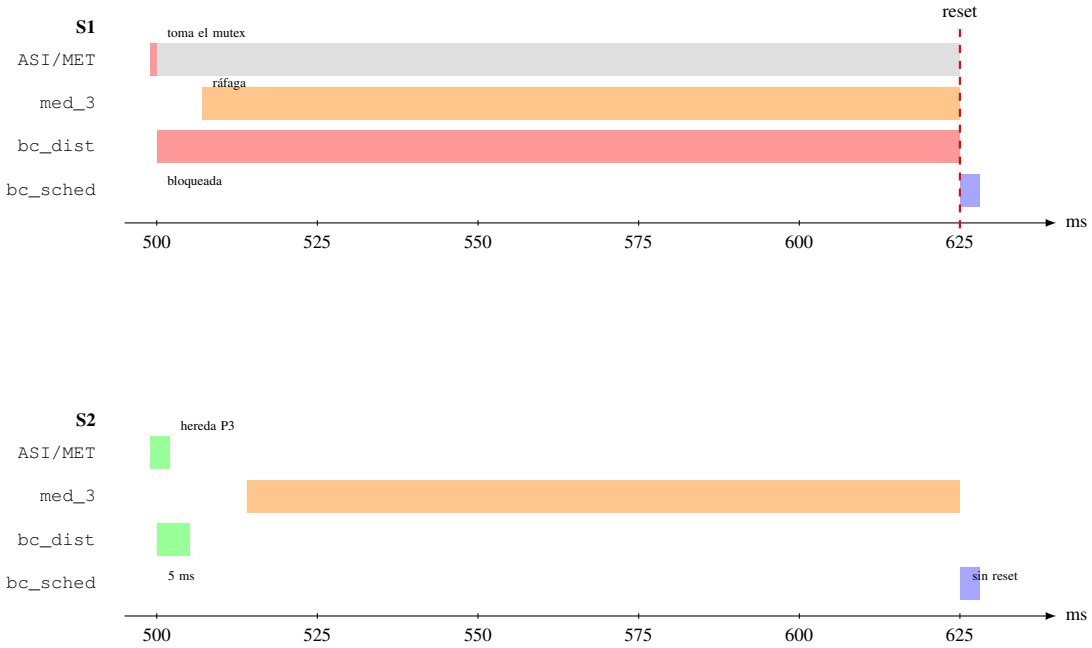


Figura 1. Timeline comparada. (a) S1: `ASI/MET` queda bloqueada dentro de la sección crítica y `bc_dist` espera 125 ms; el watchdog reinicia. (b) S2: `ASI/MET` hereda la prioridad de `bc_dist`, libera el mutex en 5 ms y el ciclo termina sin reset. Tiempos de la traza del modelo.

Tabla IV
RESULTADOS DEL MODELO DETERMINISTA (DOS SEGUNDOS SIMULADOS).

Escenario	PIP	Mutex	Reinicios	Bloqueo máx. (ms)	Fallos de plazo
S0	off	compartido	0	17,0	0
S1	off	compartido	3	125,0	3
S2	on	compartido	0	5,0	0
S3	off	separados	0	3,0	0

IV-E. Lecciones y discusión

La corrección temporal es una propiedad de configuración. El planificador de VxWorks funcionaba como estaba configurado; el fallo estaba en la configuración de un mutex. ReTiF muestra que el acceso al scheduling es un recurso que se media y se policy; la lección es que esa mediación debe incluir las opciones de sincronización, no solo las prioridades [13].

El peor caso de carga importa. El fallo no apareció con la carga nominal, sino con una carga superior a la prevista. La investigación reciente cuantifica ese costo: He et al. muestran cuánto recurso se desperdicia cuando se reserva para el peor caso [11]; Guasque et al. muestran cuánto cambia la planificabilidad cuando se modela la contención [8]. Probar solo el "mejor caso" no es probar el sistema.

La observabilidad es parte del diseño. Sin la traza, el equipo no habría podido reconstruir la secuencia. Timerlat lleva esa idea al Linux actual: mide la latencia por componentes y automatiza el análisis de causas [16]. La observabilidad no es un lujo de laboratorio; es lo que convierte un reinicio inexplicable en un diagnóstico.

La recuperación no es corrección. El watchdog reinició el sistema y preservó los datos, pero no eliminó la causa. El fallo era permanente y se repetía. Un watchdog es una red de seguridad, no un sustituto del análisis de bloqueo. Dong et al. muestran que la inversión reaparece en fronteras RTOS–GPOS y que hace falta un mecanismo explícito para acotarla [10].

Los valores por defecto del RTOS son decisiones de diseño. La opción de herencia estaba desactivada por rendimiento. Esa decisión puede ser razonable en un sistema con bloqueos cortos, pero es peligrosa cuando la sección crítica interactúa con tareas intermedias. La verificación de código de bajo nivel, como la de Manjunath y Baunach, y el aislamiento de recursos de hardware, como el de Farina et al., son complementos necesarios para auditar esos valores por defecto [14], [15]. La encuesta de Akesson et al. recuerda que la práctica industrial no siempre sigue la agenda académica; por eso las lecciones deben formularse en términos de configuración y verificación, no solo de algoritmos [5].

En conjunto, las lecciones dibujan una cadena de configuración: la política de planificación define las prioridades, la configuración del mutex define el bloqueo, la carga define si el bloqueo se manifiesta y la observabilidad define si se puede diagnosticar. Un sistema de tiempo real no es correcto porque su algoritmo sea óptimo, sino porque cada eslabón de esa cadena está auditado. Pathfinder falló en el segundo eslabón y no tenía, en el momento del incidente, una forma sistemática de auditar el primero.

V. Conclusiones

RQ1. La causa raíz fue una inversión de prioridades no acotada en el mutex interno de `select()`, creado sin herencia de prioridad. La condición habilitante fue una carga de tareas medias superior a la prevista; la condición latente fue un proceso de prueba centrado en el mejor caso y la confianza en el comportamiento por defecto de un componente comercial.

RQ2. El fallo se manifestó como un incumplimiento del ciclo de bus de 125 ms: `ASI/MET` quedó bloqueada dentro de la sección crítica, `bc_dist` esperó el mutex y `bc_sched`, al comprobar que la distribución no había terminado, ordenó un reset. El watchdog convirtió un fallo temporal en un reinicio, que recuperó el sistema pero no corrigió la causa.

RQ3. La corrección fue habilitar la herencia de prioridad en la creación del mutex. El modelo lo confirma: el bloqueo máximo baja de 125 ms a 5 ms y los reinicios desaparecen; el control sin mutex compartido también elimina los reinicios, lo que aísla la causa. La evidencia primaria —traza, réplica, diagnóstico en menos de 18 horas y análisis de Wind River— coincide con el contrafactual.

La implicación práctica es que la corrección temporal debe auditarse en tres niveles: la política de planificación, la configuración de los mecanismos de sincronización y la observabilidad del sistema. La implicación para la investigación es que los casos históricos siguen siendo útiles cuando se reconstruyen con rigor y se contrastan con un modelo ejecutable. Las limitaciones son las del modelo: no es VxWorks, no emula caché ni pipeline y no reproduce la telemetría. Como trabajo futuro se propone un barrido de carga, una sensibilidad a la duración de la sección crítica y la extensión del modelo a otros RTOS.

Referencias

[1] NASA, "Mars pathfinder," https://science.nasa.gov/mission/mars-pathfinder/, 2017, misión, aterrizaje y despliegue del rover Sojourner; consultado el 2 de noviembre de 2026.

[2] G. E. Reeves, "What really happened on mars?" https://www.cs.unc.edu/~anderson/teach/comp790/papers/mars_pathfinder_long_version.html, Dec. 1997, correo técnico del responsable del software de vuelo de Mars Pathfinder, 15 de diciembre de 1997.

[3] ——, "Priority inversion: How we found it, how we fixed it," *Dr. Dobb's Journal*, Nov. 1999, https://jacobfilipp.com/DrDobbs/articles/DDJ/1999/9911/9911a/9911as1.htm.

[4] Associated Press, "Pathfinder glitch found," The Spokesman-Review, https://www.spokesman.com/stories/1997/jul/18/pathfinder-glitch-found/, Jul. 1997, 18 de julio de 1997.

[5] B. Akesson, M. Nasri, G. Nelissen, S. Altmeyer, and R. I. Davis, "A comprehensive survey of industry practice in real-time systems," *Real-Time Systems*, vol. 58, no. 3, pp. 358–398, 2022.

[6] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[7] A. Minaeva and Z. Hanzálek, "Survey on periodic scheduling for time-triggered hard real-time systems," *ACM Computing Surveys*, vol. 54, no. 1, pp. 1–32, 2021.

[8] A. Guasque, J. M. Aceituno, P. Balbastre, J. Simó, and A. Crespo, "Schedulability analysis of dynamic priority real-time systems with contention," *The Journal of Supercomputing*, vol. 78, no. 12, pp. 14 703–14 725, 2022.

[9] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990.

[10] P. Dong, Z. Jiang, A. Burns, Y. Ding, and J. Ma, "Build real-time communication for hybrid dual-OS system," *Journal of Systems Architecture*, vol. 107, p. 101774, 2020.

[11] Q. He, Y. Sun, X. Jiang, M. Lv, J. Lee, and N. Guan, "Real-time scheduling for parallel tasks with resource reclamation," *Real-Time Systems*, vol. 60, no. 2, pp. 291–327, 2024.

[12] S. M. Salman, A. V. Papadopoulos, S. Mubeen, and T. Nolte, "Multi-processor scheduling of elastic applications in compositional real-time systems," *Journal of Systems Architecture*, vol. 122, p. 102358, 2022.

[13] G. Serra, G. Ara, P. Fara, and T. Cucinotta, "ReTiF: A declarative real-time scheduling framework for POSIX systems," *Journal of Systems Architecture*, vol. 118, p. 102210, 2021.

[14] V. Manjunath and M. Baunach, "A framework for static analysis and verification of low-level RTOS code," *Journal of Systems Architecture*, vol. 154, p. 103220, 2024.

[15] G. Farina, G. Gala, M. Cinque, and G. Fohler, "Enabling memory access isolation in real-time cloud systems using Intel's detection/regulation capabilities," *Journal of Systems Architecture*, vol. 137, p. 102848, 2023.

[16] D. B. De Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Transactions on Computers*, vol. 74, no. 8, pp. 2608–2620, 2025.